i will give you leetcode questions you have to do

1. explain the questions in simple points

2.explain the brute force approach in points with time complexity and space

3.give the code in c++ of that approach

4. explain better approach in points with time complexity and space

Done 👍

Jab bhi tu LeetCode question bhejega, main exactly iss format me answer dunga:

1. Question Explanation (Simple Language)

- Problem kya bol rahi hai
- Input kya hai
- Output kya chahiye
- Example ko simple words me explain karunga

---

2. Brute Force Approach

Idea

- Step-by-step points
- Kaise sochna hai interview me

Time Complexity

```
O(...)
```

Space Complexity

```
O(...)
```

C++ Code

```
C++
```

---

3. Better / Optimal Approach

Intuition

- Brute force ki problem kya thi
- Better approach ka main observation

Algorithm

- Point-wise steps

Time Complexity

```
O(...)
```

Space Complexity

```
O(...)
```

C++ Code

```
C++
```

---

4. Dry Run (if needed)

Main table bana ke dikhaunga:

Step	Variable	Value
1	i	0
2	j	1

Taaki DP, Two Pointer, Sliding Window, Linked List sab clearly samajh aaye.

---

5. Pattern Recognition

End me bataunga:

✅ Array
✅ Hashing
✅ Two Pointer
✅ Sliding Window
✅ Binary Search
✅ DP
✅ Greedy
✅ Graph
✅ Backtracking

Aur interview me kaise identify karna hai.

Question bhej bhai 🚀
LeetCode number ya pura statement dono chalega.

Reverse Linked List

1. Question Explanation (Simple Language)

Given the head of a linked list, reverse the list and return the new head.

Example

Input:

```
1 -> 2 -> 3 -> 4 -> 5
```

Output:

```
5 -> 4 -> 3 -> 2 -> 1
```

What we need to do?

- Every node points to the next node.
- We have to reverse all links.
- Last node will become the new head.

---

2. Brute Force Approach

Idea

- Traverse the linked list.
- Store all node values in an array/vector.
- Traverse the linked list again.
- Fill values from the vector in reverse order.

Steps

1. Store all node values in a vector.
2. Reverse the vector.
3. Traverse linked list again.
4. Replace node values using reversed vector.

Time Complexity

```
O(N)
```

Space Complexity

```
O(N)
```

(extra vector used)

C++ Code

```cpp
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        vector<int> arr;

        ListNode* temp = head;

        while(temp) {
            arr.push_back(temp->val);
            temp = temp->next;
        }

        reverse(arr.begin(), arr.end());

        temp = head;
        int i = 0;

        while(temp) {
            temp->val = arr[i++];
            temp = temp->next;
        }

        return head;
    }
};
```

---

3. Better / Optimal Approach (Iterative)

Intuition

Suppose:

```
1 -> 2 -> 3 -> NULL
```

Currently:

```
1.next = 2
2.next = 3
```

We want:

```
3 -> 2 -> 1 -> NULL
```

So for every node:

- Save next node.
- Reverse current node's pointer.
- Move ahead.

Algorithm

Initialize:

```
prev = NULL
```

‹/› C++

For every node:

```
nextNode = curr->next
curr->next = prev
prev = curr
```

‹/› C++

At the end:

‹/› C++

will point to the new head.

Dry Run

Initial:

```
NULL <- 1 -> 2 -> 3 -> NULL
prev   curr
```

Iteration 1

```
nextNode = 2
1->next = NULL
prev = 1
```

List:

```
NULL <- 1

2 -> 3
```

Iteration 2

```
nextNode = 3
2->next = 1
prev = 2
```

List:

```
NULL <- 1 <- 2

3
```

Iteration 3

```
nextNode = NULL
3->next = 2
prev = 3
```

Result:

```
3 -> 2 -> 1 -> NULL
```

Time Complexity

```
O(N)
```

Space Complexity

```
O(1)
```

C++ Code

```cpp
class Solution {
public:
    ListNode* reverseList(ListNode* head) {

        ListNode* prev = NULL;
        ListNode* curr = head;

        while(curr) {

            ListNode* nextNode = curr->next;

            curr->next = prev;

            prev = curr;
            curr = nextNode;
        }

        return prev;
    }
};
```

---

4. Recursive Approach

Idea

- Reverse the remaining list first.
- Then attach current node at the end.

Time Complexity

```
O(N)
```

Space Complexity

```
O(N)
```

(recursion stack)

C++ Code

```cpp
class Solution {
public:
    ListNode* reverseList(ListNode* head) {

        if(head == NULL || head->next == NULL)
            return head;

        ListNode* newHead = reverseList(head->next);

        head->next->next = head;
        head->next = NULL;

        return newHead;
    }
};
```

---

5. Pattern Recognition

Clues

- Linked List
- Need to change links/pointers
- No extra space required

Pattern

✅ Linked List Pointer Manipulation

Must Remember

For reversing a linked list, this template is enough:

```
prev = NULL;
curr = head;

while(curr){
    nextNode = curr->next;
    curr->next = prev;
    prev = curr;
    curr = nextNode;
}

return prev;
```

Ye Linked List ka sabse important template hai. Isse hi Reverse Linked List, Reverse in K Groups, Palindrome Linked List, Reverse Between Positions jaise questions bante hain. 🚀

Merge Two Sorted Lists

1. Question Explanation (Simple Language)

You are given heads of **two sorted linked lists**.

You need to merge them into **one sorted linked list** and return its head.

Example

Input:

```
List1: 1 -> 2 -> 4
List2: 1 -> 3 -> 4
```

Output:

```
1 -> 1 -> 2 -> 3 -> 4 -> 4
```

Key Observation

- Both lists are already sorted.
- Just like merging two sorted arrays.
- At every step, choose the smaller node.

---

2. Brute Force Approach

Idea

- Store all elements of both linked lists in a vector.
- Sort the vector.
- Create a new linked list from sorted values.

Steps

1. Traverse List1 and store values.
2. Traverse List2 and store values.
3. Sort the vector.
4. Create a new linked list using sorted values.

Time Complexity

```
O((N + M) log(N + M))
```

Space Complexity

```
O(N + M)
```

C++ Code

C++

```cpp
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {

        vector<int> arr;

        while(list1){
            arr.push_back(list1->val);
            list1 = list1->next;
```

```
        }

        while(list2){
            arr.push_back(list2->val);
            list2 = list2->next;
        }

        sort(arr.begin(), arr.end());

        ListNode* dummy = new ListNode(-1);
        ListNode* temp = dummy;

        for(int x : arr){
            temp->next = new ListNode(x);
            temp = temp->next;
        }

        return dummy->next;
    }
};
```

---

3. Better / Optimal Approach

Intuition

Since both lists are already sorted:

```
1 -> 2 -> 4

1 -> 3 -> 4
```

Compare heads:

```
1 vs 1
```

Take any one.

Then compare next nodes.

Exactly same as **Merge step of Merge Sort**.

Algorithm

1. Create a dummy node.
2. Keep a pointer `tail`.
3. Compare current nodes of both lists.
4. Attach smaller node to `tail`.
5. Move that list forward.
6. Move `tail` forward.
7. When one list ends, attach the remaining part of the other list.

Dry Run

```
List1 : 1 -> 2 -> 4
List2 : 1 -> 3 -> 4
```

Dummy:

```
-1
 ^
tail
```

Compare:

```
1 vs 1
```

Take first 1.

```
-1 -> 1
```

Compare:

```
2 vs 1
```

Take second 1.

```
-1 -> 1 -> 1
```

Compare:

```
2 vs 3
```

Take 2.

```
-1 -> 1 -> 1 -> 2
```

Compare:

```
4 vs 3
```

Take 3.

```
-1 -> 1 -> 1 -> 2 -> 3
```

Compare:

```
4 vs 4
```

Take any 4.

```
-1 -> 1 -> 1 -> 2 -> 3 -> 4 -> 4
```

Done.

Time Complexity

```
O(N + M)
```

Space Complexity

```
O(1)
```

(No extra nodes except dummy)

C++ Code

C++

```cpp
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {

        ListNode* dummy = new ListNode(-1);
        ListNode* tail = dummy;

        while(list1 && list2){

            if(list1->val <= list2->val){
                tail->next = list1;
                list1 = list1->next;
            }
            else{
                tail->next = list2;
                list2 = list2->next;
            }

            tail = tail->next;
        }

        if(list1)
            tail->next = list1;

        if(list2)
            tail->next = list2;

        return dummy->next;
    }
};
```

---

4. Recursive Approach

Idea

- Smaller node becomes answer.
- Recursively merge remaining lists.

Time Complexity

```
O(N + M)
```

Space Complexity

```
O(N + M)
```

(recursion stack)

C++ Code

```C++
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {

        if(list1 == NULL) return list2;
        if(list2 == NULL) return list1;

        if(list1->val <= list2->val){
            list1->next = mergeTwoLists(list1->next, list2);
            return list1;
        }

        list2->next = mergeTwoLists(list1, list2->next);
        return list2;
    }
};
```

---

5. Pattern Recognition

Clues

- Two sorted linked lists
- Need sorted output
- Compare current nodes only

Pattern

✅ Linked List
✅ Two Pointers
✅ Merge Sort Merge Step

Template to Remember

```C++
while(list1 && list2){

    if(list1->val <= list2->val){
        tail->next = list1;
        list1 = list1->next;
    }
    else{
        tail->next = list2;
        list2 = list2->next;
    }

    tail = tail->next;
}

tail->next = (list1 ? list1 : list2);
```

Ye template **Merge K Sorted Lists**, **Sort List (LC 148)** aur Merge Sort based Linked List questions me directly use hota hai. 🚀

Linked List cycle

1. Question Explanation (Simple Language)

Given the head of a linked list, determine whether there is a cycle in the linked list.

A cycle means that while traversing the list, you never reach `NULL` because some node points back to a previous node.

Example 1

```
3 -> 2 -> 0 -> -4
     ^         |
```

```
     |________|
```

Output:

```
true
```

Example 2

```
1 -> 2 -> NULL
```

Output:

```
false
```

What do we need to do?

- Return `true` if a cycle exists.
- Return `false` otherwise.

---

2. Brute Force Approach

Idea

While traversing the list, store every visited node's address.

If we visit the same node again, there is a cycle.

Steps

1. Create a hash set.
2. Traverse the linked list.
3. If current node already exists in the set → cycle found.
4. Otherwise insert it.
5. If traversal reaches `NULL` → no cycle.

Time Complexity

```
O(N)
```

Space Complexity

```
O(N)
```

C++ Code

```C++
class Solution {
public:
    bool hasCycle(ListNode *head) {

        unordered_set<ListNode*> st;

        while(head){

            if(st.find(head) != st.end())
                return true;

            st.insert(head);
            head = head->next;
        }

        return false;
    }
};
```

---

3. Better / Optimal Approach (Floyd's Cycle Detection)

Intuition

Use two pointers:

- Slow moves 1 step.
- Fast moves 2 steps.

If there is no cycle:

```
slow ---->
fast -------->
```

Fast will eventually reach `NULL`.

If there is a cycle:

Fast pointer will lap slow pointer and they will meet.

Algorithm

1. Initialize:

```cpp
slow = head;
fast = head;
```

2. While `fast` and `fast->next` exist:
   - Move slow by 1 step.
   - Move fast by 2 steps.
   - If they meet → cycle exists.
3. If loop ends → no cycle.

Dry Run

List:

```
1 -> 2 -> 3 -> 4 -> 5
          ^       |
          |_______|
```

Initial:

```
slow = 1
fast = 1
```

Iteration 1

```
slow = 2
fast = 3
```

Iteration 2

```
slow = 3
fast = 5
```

Iteration 3

```
slow = 4
fast = 4
```

Both meet.

Hence:

```
Cycle Exists
```

Why does it work?

Inside a cycle:

- Slow gains 1 step per iteration.
- Fast gains 2 steps per iteration.

Relative speed:

```
2 - 1 = 1 step
```

So eventually fast catches slow.

Time Complexity

```
O(N)
```

Space Complexity

```
O(1)
```

C++ Code

```C++
class Solution {
public:
    bool hasCycle(ListNode *head) {

        ListNode* slow = head;
        ListNode* fast = head;

        while(fast && fast->next){

            slow = slow->next;
            fast = fast->next->next;

            if(slow == fast)
                return true;
        }

        return false;
    }
};
```

---

4. Pattern Recognition

Clues

- Linked List
- Cycle / Loop detection
- Need O(1) space
- Asked whether loop exists

Pattern

✅ Floyd's Cycle Detection Algorithm
✅ Tortoise and Hare Algorithm

Must Remember Template

```C++
```

```
ListNode* slow = head;
ListNode* fast = head;

while(fast && fast->next){

    slow = slow->next;
    fast = fast->next->next;

    if(slow == fast)
        return true;
}

return false;
```

Interview Follow-Up

A very common follow-up is:

> "If a cycle exists, find the node where the cycle starts."

That's LeetCode 142: **Linked List Cycle II** and uses the same Floyd's algorithm with one extra trick. 🚀

reorder list

1. Question Explanation (Simple Language)

Given a linked list:

```
L0 → L1 → L2 → ... → Ln
```

Reorder it as:

```
L0 → Ln → L1 → Ln-1 → L2 → Ln-2 ...
```

Example

Input:

```
1 → 2 → 3 → 4
```

Output:

```
1 → 4 → 2 → 3
```

Input:

```
1 → 2 → 3 → 4 → 5
```

Output:

```
1 → 5 → 2 → 4 → 3
```

Important

You must **change links**, not node values.

---

2. Brute Force Approach

Idea

Store all nodes in a vector.

Then use two pointers on the vector:

```
i = 0
j = n-1
```

and connect nodes in required order.

Steps

1. Traverse list and store node pointers in vector.
2. Use `i` from start and `j` from end.
3. Connect:
   - `v[i] -> v[j]`
   - `v[j] -> v[i+1]`

4. Continue until pointers meet.
5. Set last node's next to `NULL`.

Time Complexity

```
O(N)
```

Space Complexity

```
O(N)
```

C++ Code

C++

```cpp
class Solution {
public:
    void reorderList(ListNode* head) {

        vector<ListNode*> v;

        ListNode* temp = head;

        while(temp){
            v.push_back(temp);
            temp = temp->next;
        }

        int i = 0;
        int j = v.size() - 1;

        while(i < j){

            v[i]->next = v[j];
            i++;

            if(i == j) break;

            v[j]->next = v[i];
            j--;
        }

        v[i]->next = NULL;
```

```
    }
};
```

---

3. Better / Optimal Approach

This is the famous **3-step linked list problem**.

Intuition

For:

```
1 → 2 → 3 → 4 → 5
```

We want:

```
1 → 5 → 2 → 4 → 3
```

Observation

If we can:

Step 1: Find middle

```
1 → 2 → 3 | 4 → 5
```

Step 2: Reverse second half

```
1 → 2 → 3

5 → 4
```

Step 3: Merge alternately

```
1 → 5 → 2 → 4 → 3
```

Done.

Algorithm

Step 1: Find Middle

Use slow-fast pointer.

```C++
slow = head;
fast = head;
```

When loop ends:

```
slow = middle
```

Step 2: Reverse Second Half

For:

```
4 → 5
```

becomes:

```
5 → 4
```

using Reverse Linked List.

Step 3: Merge

Take one node from first half.

Take one node from reversed second half.

Repeat.

Dry Run

Initial

```
1 → 2 → 3 → 4 → 5
```

Find Middle

```
slow = 3
```

Split:

```
1 → 2 → 3

4 → 5
```

Reverse Second Half

```
5 → 4
```

Merge

```
1 → 5

1 → 5 → 2 → 4

1 → 5 → 2 → 4 → 3
```

Done.

Time Complexity

Find Middle

```
O(N)
```

Reverse

```
O(N)
```

Merge

```
O(N)
```

Total:

```
O(N)
```

Space Complexity

```
O(1)
```

C++ Code (Optimal)

C++

```cpp
class Solution {
public:
    void reorderList(ListNode* head) {

        if(!head || !head->next)
            return;

        // Find middle
        ListNode* slow = head;
        ListNode* fast = head;

        while(fast->next && fast->next->next){
            slow = slow->next;
            fast = fast->next->next;
        }

        // Reverse second half
        ListNode* second = slow->next;
        slow->next = NULL;

        ListNode* prev = NULL;
```

```
        while(second){
            ListNode* nextNode = second->next;
            second->next = prev;
            prev = second;
            second = nextNode;
        }

        second = prev;

        // Merge alternately
        ListNode* first = head;

        while(second){

            ListNode* temp1 = first->next;
            ListNode* temp2 = second->next;

            first->next = second;
            second->next = temp1;

            first = temp1;
            second = temp2;
        }
    }
};
```

---

4. Pattern Recognition

Clues

- Linked List
- First node + Last node pattern
- Need O(1) space
- Reorder from both ends

Pattern

✅ Slow-Fast Pointer (Find Middle)

✅ Reverse Linked List

✅ Alternate Merge

Interview Trick

Whenever you see:

```
First half + Reverse(second half)
```

immediately think:

```
1. Find Middle
2. Reverse Second Half
3. Merge
```

This exact pattern is used in:

- Reorder List (LC 143)
- Palindrome Linked List (LC 234)
- Twin Sum of Linked List (LC 2130)

🚀 **Reorder List is basically a combination of 3 classic linked list templates:**

- Find Middle
- Reverse Linked List
- Merge Two Lists (alternate way)

Remove Nth Node From End of List

1. Question Explanation (Simple Language)

Given a linked list and an integer `n`:

Remove the **nth node from the end** of the list and return the head.

Example

Input:

```
head = 1 -> 2 -> 3 -> 4 -> 5
n = 2
```

From the end:

```
5 = 1st
4 = 2nd
```

Remove `4`.

Output:

```
1 -> 2 -> 3 -> 5
```

Example 2

Input:

```
1
n = 1
```

Output:

```
NULL
```

What is actually being asked?

Instead of counting from the front:

```
1 2 3 4 5
```

Count from the end:

```
5 = 1st
4 = 2nd
3 = 3rd
...
```

Remove that node.

---

2. Brute Force Approach

Idea

Find the length of the linked list first.

Then determine which node to remove from the front.

Observation

If length = `len`

and nth node from end has to be removed,

its position from start is:

```
len - n + 1
```

Steps

1. Find length of list.
2. Calculate position from start.
3. Traverse to previous node.
4. Remove target node.

Dry Run

```
1 -> 2 -> 3 -> 4 -> 5

length = 5
n = 2
```

Position:

```
5 - 2 + 1 = 4
```

Remove 4th node.

Answer:

```
1 -> 2 -> 3 -> 5
```

Time Complexity

```
O(N)
```

(Two traversals)

Space Complexity

```
O(1)
```

C++ Code

C++

```cpp
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {

        int len = 0;

        ListNode* temp = head;

        while(temp){
            len++;
            temp = temp->next;
        }

        int pos = len - n + 1;

        if(pos == 1)
            return head->next;

        temp = head;

        for(int i = 1; i < pos - 1; i++)
            temp = temp->next;

        temp->next = temp->next->next;

        return head;
    }
};
```

---

3. Better / Optimal Approach (One Pass)

Intuition

Use two pointers.

Maintain a gap of `n` nodes between them.

When fast reaches the end:

```
slow
```

will be just before the node we need to delete.

Trick

Use a **dummy node**.

Why?

Consider:

```
1
n = 1
```

After deletion:

```
NULL
```

Deleting the head becomes easy with dummy node.

Algorithm

Step 1

Create dummy.

```C++
dummy -> head
```

Step 2

Move fast pointer `n` steps ahead.

Step 3

Move both pointers together until:

```C++
fast->next == NULL
```

Step 4

Now:

```
slow
```

is before node to delete.

Remove it.

Dry Run

Input

```
1 -> 2 -> 3 -> 4 -> 5
n = 2
```

Move fast 2 steps:

```
dummy -> 1 -> 2 -> 3 -> 4 -> 5
         s         f
```

Move together:

```
dummy -> 1 -> 2 -> 3 -> 4 -> 5
                  s         f
```

Now:

```
slow = 3
```

Node after slow:

```
4
```

Delete it.

Result:

```
1 -> 2 -> 3 -> 5
```

Time Complexity

```
O(N)
```

Single traversal.

Space Complexity

```
O(1)
```

C++ Code (Optimal)

C++

```cpp
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {

        ListNode* dummy = new ListNode(0);
        dummy->next = head;

        ListNode* slow = dummy;
        ListNode* fast = dummy;

        for(int i = 0; i < n; i++)
            fast = fast->next;

        while(fast->next){

            slow = slow->next;
            fast = fast->next;
        }

        slow->next = slow->next->next;

        return dummy->next;
    }
};
```

---

4. Why Move Fast by n Steps?

Suppose:

```
1 -> 2 -> 3 -> 4 -> 5
```

Need to remove:

```
2nd from end
```

If fast starts `n` nodes ahead:

```
slow -> 1
```

```
fast -> 3
```

Now move both together.

When fast reaches last node:

```
slow -> 3
```

which is exactly before:

```
4
```

the node we need to delete.

---

5. Pattern Recognition

Clues

- kth/nth node from end
- Single pass required
- Linked List

Pattern

✅ Fast & Slow Pointer

✅ Fixed Gap Technique

Must Remember Template

```C++
ListNode* slow = dummy;
ListNode* fast = dummy;

for(int i = 0; i < n; i++)
    fast = fast->next;

while(fast->next){
    slow = slow->next;
    fast = fast->next;
```

```
}

slow->next = slow->next->next;
```

Interview Follow-Ups

Very common variations:

1. Find kth node from end.
2. Delete kth node from end.
3. Middle of Linked List.
4. Linked List Cycle.

All are based on **Fast-Slow Pointer technique**. 🚀

Copy List With Random Pointer

1. Question Explanation (Simple Language)

Each node has:

```C++
val
next
random
```

where:

- `next` → points to next node
- `random` → can point to **any node** in the list or `NULL`

Example:

```
Node1(7)  -> Node2(13) -> Node3(11)

Random pointers:

7  -> NULL
```

```
13 -> 7
11 -> 13
```

We need to create a **deep copy** of the entire list.

Deep Copy means

Create completely new nodes.

```
Original:
A -> B -> C

Copy:
A' -> B' -> C'
```

No copied node should point to any original node.

---

2. Brute Force Approach (HashMap)

Idea

For every original node, create its copy and store mapping:

```cpp
original_node -> copied_node
```

using a hashmap.

Then in a second pass:

- Set `next`
- Set `random`

using the map.

Steps

Pass 1

Create copies.

```cpp
mp[original] = copy
```

Pass 2

For every original node:

```cpp
copy->next   = mp[original->next]
copy->random = mp[original->random]
```

Example

```
Original:

A -> B -> C

Map:

A -> A'
B -> B'
C -> C'
```

Now:

```cpp
A'->next = B'
A'->random = C'
```

easy using hashmap.

Time Complexity

```
O(N)
```

Space Complexity

```
O(N)
```

(HashMap)

C++ Code

C++

```cpp
class Solution {
public:
    Node* copyRandomList(Node* head) {

        if(head == NULL)
            return NULL;

        unordered_map<Node*, Node*> mp;

        Node* temp = head;

        // Create copies
        while(temp){
            mp[temp] = new Node(temp->val);
            temp = temp->next;
        }

        temp = head;

        // Set next and random
        while(temp){

            mp[temp]->next = mp[temp->next];
            mp[temp]->random = mp[temp->random];

            temp = temp->next;
        }

        return mp[head];
    }
};
```

---

3. Better / Optimal Approach

This is the famous **O(1) space solution**.

Intuition

Instead of storing mapping in hashmap,

store copied node right beside original node.

Step 1: Insert copied nodes

Original:

```
A -> B -> C
```

After inserting copies:

```
A -> A' -> B -> B' -> C -> C'
```

Now every copy is just after its original.

Step 2: Set Random Pointers

Suppose:

```
A.random = C
```

Then:

```
A'.random = C'
```

How to find C'?

Since:

```
C'
=
C->next
```

Therefore:

```cpp
A->next->random = A->random->next
```

Magic line:

```cpp
copy->random = original->random->next
```

Step 3: Separate Lists

Current:

```
A -> A' -> B -> B' -> C -> C'
```

Need:

```
Original:
A -> B -> C

Copy:
A' -> B' -> C'
```

Detach them.

Dry Run

Original

```
1 -> 2 -> 3
```

Insert Copies

```
1 -> 1' -> 2 -> 2' -> 3 -> 3'
```

Set Random

If:

```
1.random = 3
```

then:

```C++
1'.random = 3'
```

using:

```C++
1->next->random = 1->random->next
```

Separate

Original:

```
1 -> 2 -> 3
```

Copy:

```
1' -> 2' -> 3'
```

Done.

Time Complexity

Insert copies

```
O(N)
```

Set random

```
O(N)
```

Separate

```
O(N)
```

Total:

```
O(N)
```

Space Complexity

```
O(1)
```

(No hashmap)

C++ Code (Optimal)

C++

```cpp
class Solution {
public:
    Node* copyRandomList(Node* head) {

        if(head == NULL)
            return NULL;

        // Step 1: Insert copied nodes
        Node* temp = head;

        while(temp){

            Node* copy = new Node(temp->val);
```

```
            copy->next = temp->next;
            temp->next = copy;

            temp = copy->next;
        }

        // Step 2: Set random pointers
        temp = head;

        while(temp){

            if(temp->random)
                temp->next->random = temp->random->next;

            temp = temp->next->next;
        }

        // Step 3: Separate lists
        Node* dummy = new Node(0);
        Node* copyTail = dummy;

        temp = head;

        while(temp){

            Node* copy = temp->next;

            temp->next = copy->next;

            copyTail->next = copy;
            copyTail = copy;

            temp = temp->next;
        }

        return dummy->next;
    }
};
```

---

4. Why Does `random->next` Work?

After Step 1:

```
A -> A' -> B -> B' -> C -> C'
```

Suppose:

```
A.random = C
```

Then:

```
C'
=
C->next
```

because every copy sits immediately after original.

Therefore:

```
A'.random = C'
```

becomes

```
A->next->random = A->random->next;
```

This is the key observation of the optimal solution.

---

5. Pattern Recognition

Clues

- Deep Copy
- Random Pointer
- Need O(1) Extra Space
- Mapping Original ↔ Copy

Pattern

✅ Linked List Manipulation

✅ Interweaving Nodes Technique

✅ Deep Copy Construction

Interview Tip

For **Copy List With Random Pointer**, remember the 3-step mantra:

```
1. Insert copied nodes in between originals
2. Set random pointers
3. Separate both lists
```

As soon as you hear:

```
Linked List + Random Pointer + O(1) Space
```

this pattern should immediately come to mind. 🚀

Add Two Numbers

1. Question Explanation (Simple Language)

You are given **two non-empty linked lists** representing two non-negative integers.

Digits are stored in **reverse order**.

Each node contains a single digit.

Return the sum as a linked list.

Example

Input:

```
l1 = 2 -> 4 -> 3
l2 = 5 -> 6 -> 4
```

Since digits are stored in reverse:

```
243 represents 342
564 represents 465
```

Actual addition:

```
342 + 465 = 807
```

Return:

```
7 -> 0 -> 8
```

because:

```
807 stored in reverse
```

Example 2

```
l1 = 0
l2 = 0
```

Output:

```
0
```

Example 3

```
9 -> 9 -> 9 -> 9

1
```

Addition:

```
9999 + 1 = 10000
```

Output:

```
0 -> 0 -> 0 -> 0 -> 1
```

---

2. Brute Force Approach

Idea

Convert both linked lists into numbers.

Add them.

Create a new linked list from the answer.

---

1. Traverse first list and form number.
2. Traverse second list and form number.
3. Add both numbers.
4. Create linked list from resulting number.

Why is it bad?

Suppose:

```
50+ digits
100+ digits
```

The number can exceed:

```C++
long long
```

and overflow occurs.

Therefore this approach is not valid for large inputs.

Time Complexity

```
O(N + M)
```

Space Complexity

```
O(1)
```

C++ Code (Conceptual Only)

C++

```cpp
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {

        long long num1 = 0;
        long long num2 = 0;

        long long place = 1;

        while(l1){
            num1 += l1->val * place;
            place *= 10;
            l1 = l1->next;
        }

        place = 1;

        while(l2){
            num2 += l2->val * place;
            place *= 10;
            l2 = l2->next;
        }

        long long sum = num1 + num2;

        ListNode* dummy = new ListNode(-1);
        ListNode* tail = dummy;

        if(sum == 0)
            return new ListNode(0);

        while(sum){

            tail->next = new ListNode(sum % 10);
            tail = tail->next;

            sum /= 10;
```

```
        }

        return dummy->next;
    }
};
```

⚠️ Not accepted for large test cases due to overflow.

---

3. Better / Optimal Approach

Intuition

We do addition exactly like we do on paper.

Example:

```
  342
+ 465
-----
  807
```

Digit by digit:

```
2 + 5 = 7
4 + 6 = 10
3 + 4 + carry = 8
```

Since list is already reversed:

```
2 -> 4 -> 3
5 -> 6 -> 4
```

we can directly start from the units digit.

Algorithm

Maintain:

```
‹/› C++
```

```
carry = 0
```

For every iteration:

```C++
sum = carry
```

Add current digit from list1 if available.

Add current digit from list2 if available.

Then:

```C++
digit = sum % 10
carry = sum / 10
```

Create a node with `digit`.

Continue until:

```C++
l1 == NULL
l2 == NULL
carry == 0
```

Dry Run

Input

```
2 -> 4 -> 3
5 -> 6 -> 4
```

Iteration 1

```
2 + 5 = 7

digit = 7
carry = 0
```

Answer:

```
7
```

Iteration 2

```
4 + 6 = 10

digit = 0
carry = 1
```

Answer:

```
7 -> 0
```

Iteration 3

```
3 + 4 + carry(1)

= 8
```

Answer:

```
7 -> 0 -> 8
```

Done.

Dry Run 2 (Carry at End)

```
9 -> 9 -> 9
```

```
1
```

Step 1

```
9 + 1 = 10

digit = 0
carry = 1
```

Answer:

```
0
```

Step 2

```
9 + carry = 10

digit = 0
carry = 1
```

Answer:

```
0 -> 0
```

Step 3

```
9 + carry = 10

digit = 0
carry = 1
```

Answer:

```
0 -> 0 -> 0
```

Step 4

Lists finished

carry = 1

Create extra node.

Answer:

```
0 -> 0 -> 0 -> 1
```

Time Complexity

```
O(max(N, M))
```

We traverse both lists once.

Space Complexity

```
O(max(N, M))
```

Output list is created.

(Extra auxiliary space apart from answer = O(1))

C++ Code (Optimal)

‹/› C++

```cpp
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {

        ListNode* dummy = new ListNode(-1);
        ListNode* tail = dummy;

        int carry = 0;
```

```
        while(l1 || l2 || carry){

            int sum = carry;

            if(l1){
                sum += l1->val;
                l1 = l1->next;
            }

            if(l2){
                sum += l2->val;
                l2 = l2->next;
            }

            carry = sum / 10;

            tail->next = new ListNode(sum % 10);
            tail = tail->next;
        }

        return dummy->next;
    }
};
```

---

4. Pattern Recognition

Clues

- Linked List digits
- Number addition
- Carry involved
- Digits stored in reverse order

Pattern

✅ Linked List Traversal

✅ Simulation

✅ Carry Propagation

Must Remember Template

```cpp
int carry = 0;

while(l1 || l2 || carry){

    int sum = carry;

    if(l1){
        sum += l1->val;
        l1 = l1->next;
    }

    if(l2){
        sum += l2->val;
        l2 = l2->next;
    }

    carry = sum / 10;

    tail->next = new ListNode(sum % 10);
    tail = tail->next;
}
```

Interview Follow-Ups

Very common variations:

1. **Add Two Numbers II (LC 445)**
   Digits are stored in normal order.
2. **Multiply Two Numbers Represented by Linked Lists**
3. **Sum of Linked Lists with Carry**

All are based on the same carry-handling idea. 🚀

Find The Duplicate Number

1. Question Explanation (Simple Language)

You are given an array `nums` of size `n + 1`.

Conditions

- Numbers are in the range:

```
[1, n]
```

- Only **one number is duplicated**.
- Duplicate may appear multiple times.
- You must find the duplicate number.

Example 1

Input:

```
nums = [1,3,4,2,2]
```

Output:

```
2
```

Because `2` appears twice.

Example 2

Input:

```
nums = [3,1,3,4,2]
```

Output:

```
3
```

Constraints

You are **not allowed** to modify the array.

Try to use:

```
O(1) extra space
```

---

2. Brute Force Approach

Idea

For every element, count how many times it appears.

If count > 1, return that element.

Steps

1. Pick an element.
2. Traverse entire array.
3. Count occurrences.
4. If count > 1 return it.

Time Complexity

```
O(N²)
```

Space Complexity

```
O(1)
```

C++ Code

‹/› C++

```cpp
class Solution {
public:
    int findDuplicate(vector<int>& nums) {

        int n = nums.size();
```

```
        for(int i = 0; i < n; i++){

            int cnt = 0;

            for(int j = 0; j < n; j++){

                if(nums[j] == nums[i])
                    cnt++;
            }

            if(cnt > 1)
                return nums[i];
        }

        return -1;
    }
};
```

---

3. Better Approach (Hashing)

Idea

Store frequencies using a hash map/set.

As soon as we see a number again:

```
Duplicate Found
```

Steps

1. Create a set.
2. Traverse array.
3. If element already exists → return it.
4. Else insert it.

Time Complexity

```
O(N)
```

Space Complexity

```
O(N)
```

C++ Code

C++

```cpp
class Solution {
public:
    int findDuplicate(vector<int>& nums) {

        unordered_set<int> st;

        for(int x : nums){

            if(st.find(x) != st.end())
                return x;

            st.insert(x);
        }

        return -1;
    }
};
```

---

4. Optimal Approach (Floyd's Cycle Detection)

This is the famous interview solution.

Observation

Given:

```
nums = [1,3,4,2,2]
```

Indexes:

```
0 1 2 3 4
```

Values:

```
1 3 4 2 2
```

Treat every value as a pointer to the next index.

```
0 -> 1
1 -> 3
3 -> 2
2 -> 4
4 -> 2
```

Graph:

```
0 -> 1 -> 3 -> 2 -> 4
              ^     |
              |_____|
```

A cycle is formed.

Why Cycle Exists?

Array size:

```
n + 1
```

Numbers:

```
1 to n
```

By Pigeonhole Principle:

```
One value must repeat
```

That repeated value creates a cycle.

Step 1: Find Meeting Point

Use Floyd's Algorithm.

```cpp
slow = nums[slow];
fast = nums[nums[fast]];
```

Eventually they meet.

Step 2: Find Cycle Start

Set:

```cpp
slow = nums[0];
```

Keep moving both by one step.

```cpp
slow = nums[slow];
fast = nums[fast];
```

Where they meet again = duplicate number.

Dry Run

Input

```
nums = [1,3,4,2,2]
```

Phase 1

Initial:

```
slow = 1
fast = 1
```

Move:

```
slow = nums[1] = 3
fast = nums[nums[1]]
     = nums[3]
     = 2
```

Move:

```
slow = nums[3] = 2
fast = nums[nums[2]]
     = nums[4]
     = 2
```

Meet at:

```
2
```

Phase 2

Reset:

```
slow = nums[0] = 1
fast = 2
```

Move:

```
slow = 3
fast = 4
```

Move:

```
slow = 2
fast = 2
```

Meet at:

```
2
```

Answer:

```
2
```

Time Complexity

Phase 1

```
O(N)
```

Phase 2

```
O(N)
```

Total:

```
O(N)
```

Space Complexity

```
O(1)
```

C++ Code (Optimal)

‹/› C++

```cpp
class Solution {
public:
    int findDuplicate(vector<int>& nums) {
```

```
        int slow = nums[0];
        int fast = nums[0];

        // Find intersection point
        do{
            slow = nums[slow];
            fast = nums[nums[fast]];
        }while(slow != fast);

        // Find entrance of cycle
        slow = nums[0];

        while(slow != fast){
            slow = nums[slow];
            fast = nums[fast];
        }

        return slow;
    }
};
```

---

5. Pattern Recognition

Clues

- Array size = `n + 1`
- Values range = `[1, n]`
- One duplicate exists
- O(1) space required
- Cannot modify array

Pattern

✅ Floyd's Cycle Detection (Tortoise & Hare)

Interview Trick

Whenever you see:

```
Array size = n + 1
Values = [1, n]
Find duplicate
```

```
O(1) space
Cannot modify array
```

Immediately think:

```
Value → Next Index Mapping
↓
Cycle Formation
↓
Floyd's Cycle Detection
```

This is one of the most famous "array disguised as linked list" problems in interviews. 🚀

LRU Cache

1. Question Explanation (Simple Language)

Design an LRU (Least Recently Used) Cache.

We have:

```
LRUCache(int capacity)
get(key)
put(key, value)
```

Rules

get(key)

- If key exists → return value.
- Mark it as recently used.
- If key doesn't exist → return `-1`.

---

put(key, value)

- Insert/update key-value pair.
- Mark it as recently used.
- If cache exceeds capacity:
  - Remove the Least Recently Used item.

Example

```
LRUCache cache(2);

put(1,1)
put(2,2)

get(1)
```

Returns:

```
1
```

Now order:

```
1 (most recent)
2 (least recent)
```

```
put(3,3)
```

Capacity exceeded.

Remove:

```
2
```

because it is least recently used.

Cache:

```
1,3
```

```
get(2)
```

returns

```
-1
```

---

2. Brute Force Approach

Idea

Store cache in a vector/list.

Whenever a key is used:

- Move it to front.

Whenever capacity exceeds:

- Remove last element.

For searching:

- Traverse entire list.

Operations

get()

```
Linear Search
```

put()

```
Linear Search
```

Time Complexity

get()

```
O(N)
```

put()

```
O(N)
```

Space Complexity

```
O(capacity)
```

C++ Code (Brute Force)

```cpp
class LRUCache {
public:

    int cap;
    list<pair<int,int>> cache;

    LRUCache(int capacity) {
        cap = capacity;
    }

    int get(int key) {

        for(auto it = cache.begin(); it != cache.end(); it++) {

            if(it->first == key) {

                int val = it->second;

                cache.erase(it);
                cache.push_front({key,val});

                return val;
            }
        }

        return -1;
    }

    void put(int key, int value) {

        for(auto it = cache.begin(); it != cache.end(); it++) {
```

```
            if(it->first == key) {
                cache.erase(it);
                break;
            }
        }

        cache.push_front({key,value});

        if(cache.size() > cap)
            cache.pop_back();
    }
};
```

---

3. Optimal Approach

Requirement

LeetCode asks:

```
get()  -> O(1)
put()  -> O(1)
```

HashMap alone can't tell:

```
Least Recently Used element
```

List alone can't do:

```
Search in O(1)
```

So combine both.

Data Structures Used

1. Doubly Linked List

Stores usage order.

```
Most Recent <-------> Least Recent
```

2. HashMap

Stores:

```
C++
```

so node can be found instantly.

Why Doubly Linked List?

Suppose key becomes recently used.

Need to move node to front.

In DLL:

```
Delete node -> O(1)
Insert node -> O(1)
```

because previous and next pointers are available.

Structure

Suppose:

```
capacity = 3
```

Cache:

```
head

3 <-> 1 <-> 2

tail
```

Meaning:

```
3 = Most Recently Used
2 = Least Recently Used
```

---

get(key)

Suppose:

```
get(1)
```

Found in hashmap.

Move node to front.

Before:

```
3 <-> 1 <-> 2
```

After:

```
1 <-> 3 <-> 2
```

Return value.

---

put(key,value)

Case 1: Key exists

Update value.

Move node to front.

Case 2: New key

Insert at front.

Case 3: Capacity exceeded

Remove:

```
tail->prev
```

which is least recently used.

Dry Run

Capacity = 2

---

put(1,1)

```
1
```

---

put(2,2)

```
2 <-> 1
```

---

get(1)

Move 1 to front.

```
1 <-> 2
```

---

put(3,3)

Insert:

```
3 <-> 1 <-> 2
```

Capacity exceeded.

Remove:

```
2
```

Final:

```
3 <-> 1
```

Time Complexity

get()

```
O(1)
```

HashMap lookup + DLL operations.

---

put()

```
O(1)
```

Space Complexity

```
O(capacity)
```

C++ Code (Optimal)

```cpp
class LRUCache {
public:

    int cap;

    list<pair<int,int>> dll;
```

```
    unordered_map<int, list<pair<int,int>>::iterator> mp;

    LRUCache(int capacity) {
        cap = capacity;
    }

    int get(int key) {

        if(mp.find(key) == mp.end())
            return -1;

        auto node = mp[key];

        int value = node->second;

        dll.erase(node);

        dll.push_front({key,value});

        mp[key] = dll.begin();

        return value;
    }

    void put(int key, int value) {

        if(mp.find(key) != mp.end()) {

            dll.erase(mp[key]);
        }

        dll.push_front({key,value});

        mp[key] = dll.begin();

        if(dll.size() > cap) {

            auto last = dll.back();

            mp.erase(last.first);

            dll.pop_back();
        }
    }
};
```

4. Pattern Recognition

Clues

When you hear:

```
Least Recently Used
```

or

```
Most Recently Used
```

immediately think:

```
HashMap + Doubly Linked List
```

Why?

HashMap gives:

```
Search -> O(1)
```

DLL gives:

```
Delete -> O(1)
Insert -> O(1)
Move to Front -> O(1)
```

Interview Notes

This is one of the most frequently asked system-design/data-structure questions.

Must Remember

```
HashMap
+
Doubly Linked List
```

```
=
LRU Cache
```

Similar Problems

- LRU Cache (LC 146)
- LFU Cache (LC 460)
- Design Browser History
- Design Twitter
- Design In-Memory Cache

🚀 **LRU Cache is the classic application of HashMap + Doubly Linked List.**

Merge K Sorted Lists

1. Question Explanation (Simple Language)

You are given `k` sorted linked lists.

Merge all of them into **one sorted linked list**.

Return the head of the merged list.

Example

Input:

```
lists = [
  1->4->5,
  1->3->4,
  2->6
]
```

Output:

```
1->1->2->3->4->4->5->6
```

Observation

For 2 sorted lists, we use:

```
Merge Two Sorted Lists
```

Here we have:

```
k sorted lists
```

Need one final sorted list.

---

2. Brute Force Approach

Idea

Store all values from all linked lists into an array.

Sort the array.

Create a new linked list from sorted values.

Steps

1. Traverse every linked list.
2. Push all values into a vector.
3. Sort vector.
4. Create answer linked list.

Example

```
1->4->5

1->3->4

2->6
```

Vector:

```
[1,4,5,1,3,4,2,6]
```

Sort:

```
[1,1,2,3,4,4,5,6]
```

Create linked list.

Time Complexity

Let total nodes = `N`

```
O(N log N)
```

Sorting dominates.

Space Complexity

```
O(N)
```

C++ Code

```cpp
class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*>& lists) {

        vector<int> arr;

        for(auto head : lists){

            while(head){
                arr.push_back(head->val);
                head = head->next;
            }
        }
```

```
        sort(arr.begin(), arr.end());

        ListNode* dummy = new ListNode(-1);
        ListNode* tail = dummy;

        for(int x : arr){
            tail->next = new ListNode(x);
            tail = tail->next;
        }

        return dummy->next;
    }
};
```

---

3. Better Approach (Min Heap / Priority Queue)

Intuition

At any moment, we only need the smallest node among all current heads.

Example:

```
1->4->5
1->3->4
2->6
```

Current heads:

```
1 , 1 , 2
```

Smallest:

```
1
```

Add it to answer.

Then push its next node.

Repeat.

Data Structure

Use Min Heap.

Heap stores:

```
C++
```

Algorithm

1. Insert first node of every list into min heap.
2. Extract minimum node.
3. Add it to answer.
4. Insert its next node (if exists).
5. Continue until heap becomes empty.

Dry Run

Initial heap:

```
1,1,2
```

Pop:

```
1
```

Push next:

```
4
```

Heap:

```
1,2,4
```

Pop:

```
1
```

Push:

```
3
```

Heap:

```
2,3,4
```

Continue...

Final:

```
1->1->2->3->4->4->5->6
```

Time Complexity

Total nodes = `N`

Each node:

```
Push -> O(log k)
Pop  -> O(log k)
```

Therefore:

```
O(N log k)
```

Space Complexity

```
O(k)
```

Heap stores at most one node from each list.

C++ Code

```cpp
class Solution {
public:

    struct cmp {
        bool operator()(ListNode* a, ListNode* b) {
            return a->val > b->val;
        }
    };

    ListNode* mergeKLists(vector<ListNode*>& lists) {

        priority_queue<
            ListNode*,
            vector<ListNode*>,
            cmp
        > pq;

        for(auto head : lists){
            if(head)
                pq.push(head);
        }

        ListNode* dummy = new ListNode(-1);
        ListNode* tail = dummy;

        while(!pq.empty()) {

            ListNode* node = pq.top();
            pq.pop();

            tail->next = node;
            tail = tail->next;

            if(node->next)
                pq.push(node->next);
        }

        return dummy->next;
    }
};
```

---

4. Optimal Approach (Divide & Conquer)

Intuition

This is exactly like Merge Sort.

Instead of merging all lists together:

Merge them in pairs.

Example

```
L1
L2
L3
L4
```

Round 1:

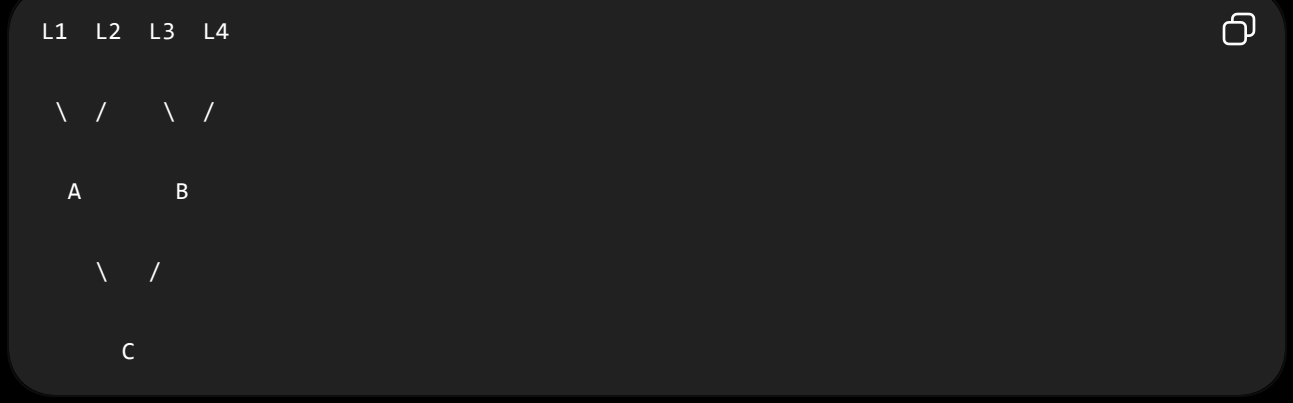
```
Merge(L1,L2) -> A
Merge(L3,L4) -> B
```

Round 2:

```
Merge(A,B)
```

Done.

Visualization



Algorithm

Repeatedly merge two lists at a time.

Use the standard:

```
Merge Two Sorted Lists
```

function.

Time Complexity

Let:

```
N = total nodes
K = number of lists
```

Number of levels:

```
log K
```

At each level:

```
O(N)
```

Total:

```
O(N log K)
```

Space Complexity

Iterative version:

```
O(1)
```

Recursive divide & conquer:

```
O(log K)
```

(recursion stack)

C++ Code (Divide & Conquer)

```
class Solution {
public:

    ListNode* mergeTwo(ListNode* a, ListNode* b) {

        ListNode dummy(-1);
        ListNode* tail = &dummy;

        while(a && b) {

            if(a->val <= b->val) {
                tail->next = a;
                a = a->next;
            }
            else {
                tail->next = b;
                b = b->next;
            }

            tail = tail->next;
        }

        tail->next = (a ? a : b);

        return dummy.next;
    }

    ListNode* mergeKLists(vector<ListNode*>& lists) {

        if(lists.empty())
            return NULL;

        int k = lists.size();

        while(k > 1) {

            int i = 0;
            int j = k - 1;

            while(i < j) {
```

```
                lists[i] = mergeTwo(lists[i], lists[j]);

                i++;
                j--;
            }

            k = (k + 1) / 2;
        }

        return lists[0];
    }
};
```

---

5. Pattern Recognition

Clues

- Multiple sorted lists
- Need smallest element repeatedly
- Merge sorted structures

Patterns

✅ Priority Queue / Min Heap

✅ Divide & Conquer

✅ Merge Sort Logic

Interview Notes

If interviewer asks quickly:

Most accepted answer:

```
Min Heap
Time = O(N log K)
Space = O(K)
```

If interviewer asks:

```
Can you do it using Merge Sort idea?
```

Use:

```
Divide & Conquer
Time = O(N log K)
Space = O(1) iterative
```

Must Remember

For:

```
Merge K Sorted Lists
```

Think immediately:

```
1. Min Heap (most common)

OR

2. Divide & Conquer (merge sort style)
```

Both achieve:

```
O(N log K)
```

which is the target optimal complexity. 🚀

Reverse Nodes In K Group

1. Question Explanation (Simple Language)

Given a linked list and an integer `k`.

Reverse the nodes of the linked list **k at a time**.

If the number of remaining nodes is less than `k`, leave them as they are.

Example 1

Input:

```
head = 1 -> 2 -> 3 -> 4 -> 5
k = 2
```

Output:

```
2 -> 1 -> 4 -> 3 -> 5
```

Explanation:

```
(1,2) reversed -> 2,1
(3,4) reversed -> 4,3
5 remains same
```

Example 2

Input:

```
head = 1 -> 2 -> 3 -> 4 -> 5
k = 3
```

Output:

```
3 -> 2 -> 1 -> 4 -> 5
```

Explanation:

```
(1,2,3) reversed
Only 2 nodes remain
So leave them unchanged
```

---

2. Brute Force Approach

Idea

Store all nodes in a vector.

Reverse every block of size `k`.

Reconnect the nodes.

Steps

1. Store all node pointers in vector.
2. Traverse vector in chunks of size `k`.
3. Reverse each valid chunk.
4. Rebuild linked list.

Example

```
1 2 3 4 5 6
k = 3
```

Vector:

```
[1,2,3,4,5,6]
```

Reverse chunks:

```
[3,2,1,6,5,4]
```

Rebuild list.

Time Complexity

```
O(N)
```

Space Complexity

```
O(N)
```

(vector)

C++ Code

```cpp
class Solution {
public:
    ListNode* reverseKGroup(ListNode* head, int k) {

        vector<ListNode*> nodes;

        ListNode* temp = head;

        while(temp){
            nodes.push_back(temp);
            temp = temp->next;
        }

        int n = nodes.size();

        for(int i = 0; i + k <= n; i += k){
            reverse(nodes.begin() + i,
                    nodes.begin() + i + k);
        }

        for(int i = 0; i < n - 1; i++)
            nodes[i]->next = nodes[i + 1];

        if(n)
            nodes[n - 1]->next = NULL;

        return n ? nodes[0] : NULL;
    }
};
```

---

3. Better / Optimal Approach

Intuition

This problem is basically:

```
Reverse Linked List
+
Do it in groups of k
```

For:

```
1 -> 2 -> 3 -> 4 -> 5 -> 6
k = 3
```

We want:

```
3 -> 2 -> 1 -> 6 -> 5 -> 4
```

Main Observation

Before reversing a group:

Check whether `k` nodes exist.

If not:

```
Leave them unchanged
```

Algorithm

Step 1

Find kth node from current position.

Suppose:

```
1 -> 2 -> 3 -> 4 -> 5
```

k = 3

kth node:

```
3
```

Step 2

Temporarily cut the group.

```
1 -> 2 -> 3 -> NULL

4 -> 5
```

Step 3

Reverse first part.

```
3 -> 2 -> 1
```

Step 4

Attach it back.

```
3 -> 2 -> 1 -> 4 -> 5
```

Step 5

Repeat from next group.

Dry Run

Input

```
1 -> 2 -> 3 -> 4 -> 5
k = 2
```

Group 1

```
1 -> 2
```

Reverse:

```
2 -> 1
```

Remaining:

```
3 -> 4 -> 5
```

Group 2

```
3 -> 4
```

Reverse:

```
4 -> 3
```

Result:

```
2 -> 1 -> 4 -> 3 -> 5
```

Group 3

Only:

```
5
```

Less than `k`.

Keep it unchanged.

Time Complexity

Each node is visited a constant number of times.

```
O(N)
```

Space Complexity

```
O(1)
```

C++ Code (Optimal)

```cpp
class Solution {
public:

    ListNode* reverseList(ListNode* head) {

        ListNode* prev = NULL;
        ListNode* curr = head;

        while(curr){

            ListNode* nextNode = curr->next;

            curr->next = prev;

            prev = curr;
            curr = nextNode;
        }

        return prev;
    }

    ListNode* getKthNode(ListNode* curr, int k) {

        while(curr && --k)
            curr = curr->next;

        return curr;
```

```
    }

    ListNode* reverseKGroup(ListNode* head, int k) {

        ListNode* temp = head;
        ListNode* prevGroupTail = NULL;

        while(temp){

            ListNode* kthNode = getKthNode(temp, k);

            if(kthNode == NULL){

                if(prevGroupTail)
                    prevGroupTail->next = temp;

                break;
            }

            ListNode* nextGroup = kthNode->next;

            kthNode->next = NULL;

            ListNode* newHead = reverseList(temp);

            if(temp == head)
                head = newHead;
            else
                prevGroupTail->next = newHead;

            prevGroupTail = temp;

            temp->next = nextGroup;

            temp = nextGroup;
        }

        return head;
    }
};
```

---

4. Even Cleaner Recursive Solution

Idea

1. Check if k nodes exist.

2. Reverse first k nodes.
3. Recursively solve remaining list.
4. Attach answers.

Time Complexity

```
O(N)
```

Space Complexity

```
O(N/k)
```

(recursion stack)

C++ Code

```cpp
class Solution {
public:
    ListNode* reverseKGroup(ListNode* head, int k) {

        ListNode* temp = head;

        for(int i = 0; i < k; i++) {

            if(temp == NULL)
                return head;

            temp = temp->next;
        }

        ListNode* prev = NULL;
        ListNode* curr = head;

        for(int i = 0; i < k; i++) {

            ListNode* nextNode = curr->next;

            curr->next = prev;
```

```
            prev = curr;
            curr = nextNode;
        }

        head->next = reverseKGroup(curr, k);

        return prev;
    }
};
```

---

5. Pattern Recognition

Clues

- Reverse every k nodes
- Linked List
- Reverse chunks/groups

Pattern

✅ Reverse Linked List

✅ Find Kth Node

✅ Linked List Segmentation

Must Remember

This problem is built from **two classic templates**:

Template 1

```
</> C++
```

Template 2

```
</> C++
```

Combine them:

```
Find K Nodes
↓
Cut Group
↓
Reverse
↓
Reconnect
↓
Repeat
```

Interview Tip

Many students try to memorize this entire problem.

Instead remember:

```
Reverse K Group
=
Reverse Linked List
+
Find Kth Node
+
Reconnect Groups
```

Once you're comfortable with these three templates, LC 25 becomes much easier to derive in an interview. 🚀